Chicago Weather Trend Analytics Project

Developer: Nivin Varghese, MSBA 2026
Target Audience: Chicago city tourists and residents

1. Introduction / Background

Weather data plays an important role in transportation, tourism, agriculture, public safety, and everyday decision-making. Access to organized and visualized weather information can help individuals and organizations better understand climate patterns, monitor precipitation trends, and prepare for changing weather conditions.
This project focuses on collecting and analyzing weather data for Chicago using the Open-Meteo API. Chicago experiences significant seasonal weather changes, including temperature fluctuations, rainfall, snowfall, and storms, making it an ideal location for weather trend analysis.
The project will build an end-to-end data engineering pipeline that retrieves weather data from a REST API, processes and stores the data, and presents insights through an interactive dashboard.
This analysis may benefit:

- Tourists and travelers planning visits and outdoor activities in Chicago based on weather conditions
- Transportation and traffic management agencies monitoring weather impacts on road safety and travel conditions
- Agricultural planners and environmental researchers studying precipitation and seasonal climate patterns
- Chicago residents who want easier access to historical weather trends and future forecasts for daily planning

From a technical perspective, this project will demonstrate practical skills in REST API integration, automated data collection, data cleaning and transformation, database storage, and interactive dashboard visualization.

2. Problem Statement

Weather information is commonly provided through REST APIs in raw JSON format, which can be difficult for users to organize, analyze, and interpret for long-term trend analysis. Although weather data is widely available, there is often no centralized and user-friendly system that transforms this raw data into meaningful insights and visual reports.

3. Objectives (Specific Aims)

The primary objective of this project is to design and develop an end-to-end weather data pipeline for Chicago using the Open-Meteo REST API.
The project aims to:

- Extract daily historical and forecast weather data from the Open-Meteo API using Python
- Retrieve weather variables such as maximum temperature, minimum temperature, precipitation totals, and precipitation probability
- Transform and clean JSON weather data into a structured tabular format using Pandas
- Store processed weather data in CSV files and SQL
- Develop an interactive dashboard using Dash or Power BI for weather visualization and analysis
- Provide weather insights that may support tourism planning, traffic and transportation management, agriculture monitoring, and daily planning for residents

4. Methodology / Technical Approach

This project will follow an ETL (Extract, Transform, Load) workflow to collect, process, store, and visualize Chicago weather data obtained from the Open-Meteo REST API.
Data Source / API : Open-Meteo API : https://open-meteo.com

Location : Chicago, Illinois
Latitude: 41.85, Longitude: -87.65

Weather variables to be collected:

- Maximum temperature
- Minimum temperature
- Precipitation totals
- Precipitation probability

The API returns weather information in JSON format, which will be processed using Python.

Tools and Technologies

Component	Technology
Programming Language	Python
API Requests	Open Meteo / JSON
Data Processing	Pandas
Database / Storage	SQL
Data Visualization	Dash or Power BI
Version Control	Git/GitHub

ETL Workflow Design

Extract

- Use Python and the requests library to send API requests to Open-Meteo
- Retrieve historical and forecast weather data in JSON format
- Automate API calls for periodic data updates

Transform

- JSON responses are transformed into structured dataframe
- Use Pandas for:
  - Data cleaning
  - Handling missing values

Load

- Store cleaned data in:
  - CSV files
  - SQL database for querying and analysis
- Organize weather data into structured tables for efficient retrieval

Visualization Strategy

The final dashboard will be developed using:

- Dash by Plotly or
- Microsoft Power BI

Planned visualizations include:

- Line Chart – Daily Temperature Trends
- Area Chart – Temperature Range
- Bar Chart – Daily Precipitation Totals
- Scatter Plot – Temperature vs Precipitation

ETL Architecture Diagram

Open-Meteo REST API → Python Requests → JSON Weather Data → Pandas Data Cleaning → CSV/SQL Storage → Dash or Power BI Dashboard → Interactive Weather Insights

6. Timeline

The project will be completed over five weeks following the ETL workflow approach.

Week 1 : Research Open-Meteo API, define project scope, retrieve sample weather data

Week 2 : Set up Python environment, build API extraction pipeline using requests, validate API responses

Week 3 : Clean and preprocess data, handle missing values, store data in SQL/CSV format

Week 4 : Develop Dash or Power BI dashboard, create charts/KPIs, add filters and interactivity

Week 5 : Final testing and debugging, refine dashboard, document workflow, prepare final report and presentation

7. Expected Outcomes

The completed project will produce:

- A working ETL pipeline for Chicago weather data
- Automated API data extraction and processing scripts
- Structured weather datasets stored in CSV/database format
- An interactive dashboard displaying weather trends and forecasts
- Visual insights related to tourism, transportation, agriculture, and daily weather monitoring
- A final analytical report and project presentation demonstrating the complete workflow and findings

The project will demonstrate practical skills in API integration, ETL development, database management, and interactive data visualization.

Reference link:
https://open-meteo.com/en/docs?latitude=41.85&longitude=-87.65&forecast_days=1&past_days=31&timezone=auto&bounding_box=-90,-180,90,180&hourly=&daily=temperature_2m_max,temperature_2m_min,precipitation_sum,precipitation_probability_max&temperature_unit=fahrenheit&wind_speed_unit=mph&precipitation_unit=inch#location_and_time